

BD POSTGRESQL

-- Crear las 5 bases de datos

CREATE DATABASE optica_seguridad;

CREATE DATABASE optica_catalogo;

CREATE DATABASE optica_carrito;

CREATE DATABASE optica_ventas;

CREATE DATABASE optica_citas;

-- =====

-- **SCRIPT PARA: optica_seguridad**

-- =====

-- Crear tipo ENUM

CREATE TYPE tipo_documento_enum AS ENUM ('DNI', 'Pasaporte', 'Carnet Extranjeria');

-- Tabla de Roles

```
CREATE TABLE roles (  
    id_rol SERIAL PRIMARY KEY,  
    nombre_rol VARCHAR(50) NOT NULL UNIQUE,  
    descripcion VARCHAR(255),  
    estado BOOLEAN DEFAULT TRUE,  
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Usuarios

```
CREATE TABLE usuarios (  
    id_usuario SERIAL PRIMARY KEY,  
    nombres VARCHAR(100) NOT NULL,  
    apellidos VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    telefono VARCHAR(20),  
    documento_identidad VARCHAR(20) UNIQUE,  
    tipo_documento tipo_documento_enum DEFAULT 'DNI',  
    fecha_nacimiento DATE,  
    id_rol INT NOT NULL,  
    estado BOOLEAN DEFAULT TRUE,  
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    ultimo_acceso TIMESTAMP NULL,  
    token_recuperacion VARCHAR(255) NULL,  
    token_expiracion TIMESTAMP NULL,  
    FOREIGN KEY (id_rol) REFERENCES roles(id_rol)  
);
```

-- Índices

```
CREATE INDEX idx_usuarios_email ON usuarios(email);
```

```
-- Datos iniciales
```

```
INSERT INTO roles (nombre_rol, descripcion) VALUES  
( 'ROLE_ADMIN', 'Administrador del sistema'),  
( 'ROLE_CLIENTE', 'Cliente de la óptica'),  
( 'ROLE_EMPLEADO', 'Empleado de la óptica');
```

```
-- Usuario Admin (password: admin123)
```

```
INSERT INTO usuarios (nombres, apellidos, email, password, documento_identidad, id_rol)  
VALUES  
( 'Admin', 'Sistema', 'admin@visum.com',  
'$2a$10$N9qo8uLOickgx2ZMRZoMye1J8bOFCTGZI4PvvLtGxQkLYLGZQkYpO',  
'12345678', 1);
```

```
-- Verificar
```

```
SELECT * FROM roles;  
SELECT * FROM usuarios;
```

```
-- =====
```

```
-- SCRIPT PARA: optica_catalogo
```

```
-- =====
```

```
-- Crear tipo ENUM
```

```
CREATE TYPE genero_enum AS ENUM ('Hombre', 'Mujer', 'Unisex');
```

```
-- Tabla de Categorías
```

```
CREATE TABLE categorias (  
  id_categoria SERIAL PRIMARY KEY,  
  nombre_categoria VARCHAR(100) NOT NULL,  
  descripcion TEXT,  
  imagen_url VARCHAR(255),  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Tabla de Marcas
```

```
CREATE TABLE marcas (  
  id_marca SERIAL PRIMARY KEY,  
  nombre_marca VARCHAR(100) NOT NULL UNIQUE,  
  descripcion TEXT,  
  logo_url VARCHAR(255),  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Tabla de Productos
```

```

CREATE TABLE productos (
  id_producto SERIAL PRIMARY KEY,
  nombre_producto VARCHAR(200) NOT NULL,
  descripcion TEXT,
  id_categoria INT NOT NULL,
  id_marca INT NOT NULL,
  precio_unitario DECIMAL(10, 2) NOT NULL,
  precio_descuento DECIMAL(10, 2) NULL,
  stock INT NOT NULL DEFAULT 0,
  color VARCHAR(50),
  forma VARCHAR(50),
  material VARCHAR(100),
  genero genero_enum DEFAULT 'Unisex',
  es_destacado BOOLEAN DEFAULT FALSE,
  es_nuevo BOOLEAN DEFAULT FALSE,
  en_promocion BOOLEAN DEFAULT FALSE,
  estado BOOLEAN DEFAULT TRUE,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria),
  FOREIGN KEY (id_marca) REFERENCES marcas(id_marca)
);

```

-- Función y Trigger

```

CREATE OR REPLACE FUNCTION actualizar_fecha_modificacion()
RETURNS TRIGGER AS $$
BEGIN
  NEW.fecha_actualizacion = CURRENT_TIMESTAMP;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trigger_actualizar_productos
BEFORE UPDATE ON productos
FOR EACH ROW
EXECUTE FUNCTION actualizar_fecha_modificacion();

```

-- Tabla de Imágenes

```

CREATE TABLE imagenes_producto (
  id_imagen SERIAL PRIMARY KEY,
  id_producto INT NOT NULL,
  url_imagen VARCHAR(255) NOT NULL,
  es_principal BOOLEAN DEFAULT FALSE,
  orden INT DEFAULT 0,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_producto) REFERENCES productos(id_producto) ON DELETE
CASCADE
);

```

-- Tabla de Banners

```
CREATE TABLE banners (  
  id_banner SERIAL PRIMARY KEY,  
  titulo VARCHAR(200) NOT NULL,  
  descripcion TEXT,  
  imagen_url VARCHAR(255) NOT NULL,  
  enlace VARCHAR(255),  
  orden INT DEFAULT 0,  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_inicio DATE,  
  fecha_fin DATE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Reseñas

```
CREATE TABLE resenas_producto (  
  id_resena SERIAL PRIMARY KEY,  
  id_producto INT NOT NULL,  
  id_usuario INT NOT NULL,  
  calificacion INT NOT NULL CHECK (calificacion BETWEEN 1 AND 5),  
  comentario TEXT,  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_producto) REFERENCES productos(id_producto) ON DELETE  
  CASCADE  
);
```

-- Tabla de Favoritos

```
CREATE TABLE favoritos (  
  id_favorito SERIAL PRIMARY KEY,  
  id_usuario INT NOT NULL,  
  id_producto INT NOT NULL,  
  fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (id_producto) REFERENCES productos(id_producto) ON DELETE  
  CASCADE,  
  UNIQUE (id_usuario, id_producto)  
);
```

-- Índices

```
CREATE INDEX idx_productos_categoria ON productos(id_categoria);  
CREATE INDEX idx_productos_marca ON productos(id_marca);  
CREATE INDEX idx_productos_precio ON productos(precio_unitario);  
CREATE INDEX idx_productos_estado ON productos(estado);  
CREATE INDEX idx_productos_busqueda ON productos  
USING GIN (to_tsvector('spanish', nombre_producto || ' ' || COALESCE(descripcion, '')));
```

-- Datos iniciales

```

INSERT INTO categorias (nombre_categoria, descripcion) VALUES
('Monturas Ópticas', 'Monturas para lentes de medida'),
('Lentes de Sol', 'Gafas de sol con protección UV'),
('Lentes de Contacto', 'Lentes de contacto blandos y rígidos'),
('Accesorios', 'Estuches, paños y productos de limpieza');

```

```

INSERT INTO marcas (nombre_marca, descripcion) VALUES
('Ray-Ban', 'Marca icónica de gafas de sol'),
('Oakley', 'Gafas deportivas de alta calidad'),
('Prada', 'Diseño italiano de lujo'),
('Guess', 'Moda y estilo contemporáneo'),
('Arnette', 'Estilo urbano y juvenil');

```

-- Productos de ejemplo

```

INSERT INTO productos (nombre_producto, descripcion, id_categoria, id_marca,
precio_unitario, stock, color, genero, es_destacado) VALUES
('Ray-Ban Aviator Classic', 'Clásicas gafas de aviador con lentes polarizadas', 2, 1, 450.00,
15, 'Dorado', 'Unisex', true),
('Oakley Frogskins', 'Estilo retro con tecnología moderna', 2, 2, 380.00, 20, 'Negro', 'Unisex',
true),
('Prada PR 17WS', 'Elegancia italiana en cada detalle', 2, 3, 890.00, 8, 'Havana', 'Mujer',
false);

```

-- Verificar

```

SELECT * FROM categorias;
SELECT * FROM marcas;
SELECT * FROM productos;

```

```

-- =====
-- SCRIPT PARA: optica_carrito
-- =====

```

-- Crear tipo ENUM

```

CREATE TYPE estado_carrito_enum AS ENUM ('Activo', 'Completado', 'Abandonado');

```

-- Función para actualizar fecha

```

CREATE OR REPLACE FUNCTION actualizar_fecha_modificacion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.fecha_actualizacion = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

-- Tabla de Carritos

```

CREATE TABLE carritos (
    id_carrito SERIAL PRIMARY KEY,

```

```

    id_usuario INT NOT NULL,
    estado estado_carrito_enum DEFAULT 'Activo',
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER trigger_actualizar_carritos
BEFORE UPDATE ON carritos
FOR EACH ROW
EXECUTE FUNCTION actualizar_fecha_modificacion();

-- Tabla de Detalle del Carrito
CREATE TABLE detalle_carrito (
    id_detalle SERIAL PRIMARY KEY,
    id_carrito INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10, 2) NOT NULL,
    subtotal DECIMAL(10, 2) NOT NULL,
    fecha_agregado TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_carrito) REFERENCES carritos(id_carrito) ON DELETE CASCADE
);

-- Índices
CREATE INDEX idx_carritos_usuario ON carritos(id_usuario);
CREATE INDEX idx_detalle_carrito ON detalle_carrito(id_carrito);

-- Verificar
SELECT * FROM carritos;
SELECT * FROM detalle_carrito;

-- =====
-- SCRIPT PARA: optica_ventas
-- =====

-- Crear tipos ENUM
CREATE TYPE tipo_entrega_enum AS ENUM ('Envio', 'Recojo en Tienda');
CREATE TYPE estado_pedido_enum AS ENUM ('Pendiente', 'Confirmado', 'En Preparacion', 'Enviado', 'Entregado', 'Cancelado');
CREATE TYPE estado_pago_enum AS ENUM ('Pendiente', 'Pagado', 'Fallido', 'Reembolsado');

-- Función para actualizar fecha
CREATE OR REPLACE FUNCTION actualizar_fecha_modificacion()
RETURNS TRIGGER AS $$
BEGIN
    NEW.fecha_actualizacion = CURRENT_TIMESTAMP;
    RETURN NEW;

```

END;

\$\$ LANGUAGE plpgsql;

-- Tabla de Direcciones de Envío

```
CREATE TABLE direcciones_envio (  
  id_direccion SERIAL PRIMARY KEY,  
  id_usuario INT NOT NULL,  
  nombre_destinatario VARCHAR(150) NOT NULL,  
  telefono_destinatario VARCHAR(20) NOT NULL,  
  departamento VARCHAR(100) NOT NULL,  
  provincia VARCHAR(100) NOT NULL,  
  distrito VARCHAR(100) NOT NULL,  
  direccion_linea1 VARCHAR(255) NOT NULL,  
  direccion_linea2 VARCHAR(255),  
  referencia TEXT,  
  codigo_postal VARCHAR(10),  
  es_principal BOOLEAN DEFAULT FALSE,  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Métodos de Pago

```
CREATE TABLE metodos_pago (  
  id_metodo_pago SERIAL PRIMARY KEY,  
  nombre_metodo VARCHAR(100) NOT NULL,  
  descripcion TEXT,  
  icono_url VARCHAR(255),  
  estado BOOLEAN DEFAULT TRUE,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Tabla de Pedidos

```
CREATE TABLE pedidos (  
  id_pedido SERIAL PRIMARY KEY,  
  numero_pedido VARCHAR(20) NOT NULL UNIQUE,  
  id_usuario INT NOT NULL,  
  id_direccion_envio INT NULL,  
  tipo_entrega tipo_entrega_enum NOT NULL,  
  id_sucursal_recojo INT NULL,  
  id_metodo_pago INT NOT NULL,  
  subtotal DECIMAL(10, 2) NOT NULL,  
  costo_envio DECIMAL(10, 2) DEFAULT 0.00,  
  descuento DECIMAL(10, 2) DEFAULT 0.00,  
  total DECIMAL(10, 2) NOT NULL,  
  estado_pedido estado_pedido_enum DEFAULT 'Pendiente',  
  estado_pago estado_pago_enum DEFAULT 'Pendiente',  
  notas TEXT,  
  fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_direccion_envio) REFERENCES direcciones_envio(id_direccion),
    FOREIGN KEY (id_metodo_pago) REFERENCES metodos_pago(id_metodo_pago)
);

CREATE TRIGGER trigger_actualizar_pedidos
BEFORE UPDATE ON pedidos
FOR EACH ROW
EXECUTE FUNCTION actualizar_fecha_modificacion();

-- Tabla de Detalle de Pedidos
CREATE TABLE detalle_pedido (
    id_detalle_pedido SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad INT NOT NULL,
    precio_unitario DECIMAL(10, 2) NOT NULL,
    subtotal DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido) ON DELETE CASCADE
);

-- Índices
CREATE INDEX idx_pedidos_usuario ON pedidos(id_usuario);
CREATE INDEX idx_pedidos_estado ON pedidos(estado_pedido);
CREATE INDEX idx_direcciones_usuario ON direcciones_envio(id_usuario);

-- Datos iniciales
INSERT INTO metodos_pago (nombre_metodo, descripcion) VALUES
('Tarjeta de Crédito/Débito', 'Visa, Mastercard, American Express'),
('Yape', 'Pago mediante Yape'),
('Plin', 'Pago mediante Plin'),
('Transferencia Bancaria', 'Transferencia directa'),
('Pago Contra Entrega', 'Pago en efectivo al recibir');

-- Verificar
SELECT * FROM metodos_pago;
SELECT * FROM pedidos;

-- =====
-- SCRIPT PARA: optica_citas
-- =====

-- Crear tipo ENUM
CREATE TYPE estado_cita_enum AS ENUM ('Programada', 'Confirmada', 'Completada',
'Cancelada', 'No Asistio');

-- Función para actualizar fecha
CREATE OR REPLACE FUNCTION actualizar_fecha_modificacion()

```

```
RETURNS TRIGGER AS $$
BEGIN
    NEW.fecha_actualizacion = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

-- Tabla de Sucursales

```
CREATE TABLE sucursales (
    id_sucursal SERIAL PRIMARY KEY,
    nombre_sucursal VARCHAR(150) NOT NULL,
    direccion VARCHAR(255) NOT NULL,
    departamento VARCHAR(100) NOT NULL,
    provincia VARCHAR(100) NOT NULL,
    distrito VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    email VARCHAR(150),
    latitud DECIMAL(10, 8),
    longitud DECIMAL(11, 8),
    horario_atencion JSONB,
    estado BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- Tabla de Tipos de Servicios

```
CREATE TABLE tipos_servicio (
    id_tipo_servicio SERIAL PRIMARY KEY,
    nombre_servicio VARCHAR(150) NOT NULL,
    descripcion TEXT,
    duracion_minutos INT NOT NULL DEFAULT 30,
    precio DECIMAL(10, 2) DEFAULT 0.00,
    estado BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- Tabla de Citas

```
CREATE TABLE citas (
    id_cita SERIAL PRIMARY KEY,
    numero_cita VARCHAR(20) NOT NULL UNIQUE,
    id_usuario INT NOT NULL,
    id_sucursal INT NOT NULL,
    id_tipo_servicio INT NOT NULL,
    fecha_cita DATE NOT NULL,
    hora_cita TIME NOT NULL,
    estado_cita estado_cita_enum DEFAULT 'Programada',
    notas TEXT,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (id_sucursal) REFERENCES sucursales(id_sucursal),
FOREIGN KEY (id_tipo_servicio) REFERENCES tipos_servicio(id_tipo_servicio)
);
```

```
CREATE TRIGGER trigger_actualizar_citas
BEFORE UPDATE ON citas
FOR EACH ROW
EXECUTE FUNCTION actualizar_fecha_modificacion();
```

-- Índices

```
CREATE INDEX idx_citas_usuario ON citas(id_usuario);
CREATE INDEX idx_citas_fecha ON citas(fecha_cita);
CREATE INDEX idx_citas_sucursal ON citas(id_sucursal);
```

-- Datos iniciales

```
INSERT INTO tipos_servicio (nombre_servicio, descripcion, duracion_minutos, precio)
VALUES
```

```
('Examen de Vista', 'Evaluación completa de la visión', 30, 30.00),
('Ajuste de Montura', 'Ajuste y calibración de monturas', 15, 0.00),
('Limpieza de Lentes', 'Limpieza profunda profesional', 10, 0.00),
('Graduación de Lentes', 'Medición para lentes de medida', 45, 50.00);
```

```
INSERT INTO sucursales (nombre_sucursal, direccion, departamento, provincia, distrito,
telefono, email, horario_atencion) VALUES
```

```
('VISUM - Centro de Lima', 'Jr. de la Unión 500', 'Lima', 'Lima', 'Lima', '(01) 234-5678',
'centrolima@visum.com',
'{"lunes": "9:00-20:00", "martes": "9:00-20:00", "miercoles": "9:00-20:00", "jueves":
"9:00-20:00", "viernes": "9:00-20:00", "sabado": "9:00-20:00", "domingo":
"10:00-18:00"}::JSONB'),
('VISUM - Miraflores', 'Av. Larco 1234', 'Lima', 'Lima', 'Miraflores', '(01) 234-5679',
'miraflores@visum.com',
'{"lunes": "9:00-21:00", "martes": "9:00-21:00", "miercoles": "9:00-21:00", "jueves":
"9:00-21:00", "viernes": "9:00-21:00", "sabado": "9:00-21:00", "domingo":
"10:00-19:00"}::JSONB'),
('VISUM - San Isidro', 'Av. Javier Prado 890', 'Lima', 'Lima', 'San Isidro', '(01) 234-5680',
'sanisisidro@visum.com',
'{"lunes": "9:00-20:00", "martes": "9:00-20:00", "miercoles": "9:00-20:00", "jueves":
"9:00-20:00", "viernes": "9:00-20:00", "sabado": "9:00-20:00", "domingo":
"10:00-18:00"}::JSONB');
```

-- Verificar

```
SELECT * FROM tipos_servicio;
SELECT * FROM sucursales;
SELECT * FROM citas;
```

